

Natural Language Processing

CS4063

ASSIGNMENT 3



National University
of computer and emerging sciences

SUBMITTED BY:

Muhammad Moiz Khalid
23i-2552

BS-DS(6C)

DATE:

29th April 2026

Github Link: <https://github.com/moizkhalidd/i23-2552-NLP-Assignment3>

Overview

This report documents the design, implementation, and evaluation of a three-stage NLP pipeline built on the Amazon Reviews dataset. The system performs sentiment understanding through an encoder-only Transformer, retrieves semantically similar examples using learned embeddings, and generates natural language explanations through a decoder-only Transformer.

The pipeline flows as follows: the encoder processes a review and produces a fixed-dimensional embedding along with sentiment and length classifications. These embeddings are stored and used at inference time to retrieve similar training reviews. The decoder then takes the original review, its predicted labels, and the retrieved context to generate a short natural language explanation of the predicted sentiment.

Dataset Construction

The dataset was constructed from three Amazon Reviews product categories: Cell Phones and Accessories, Beauty, and Home and Kitchen. Twelve thousand reviews were loaded from each category, yielding a total of 36,000 samples. Each sample consists of the raw review text and its corresponding star rating on a scale of one to five. This selection satisfies the assignment requirement of three or more categories with ten to fifteen thousand samples each and a total between thirty thousand and forty-five thousand.

The full dataset was shuffled with a fixed random seed of 42 to ensure reproducibility. It was then divided into training, validation, and test splits at 70%, 15%, and 15% respectively, producing approximately 25,200 training samples, 5,400 validation samples, and 5,400 test samples.

```
Loaded 12000 records from cellphones.json
Loaded 12000 records from beauty.json
Loaded 12000 records from home.json

Total records loaded: 36000
```

```
Train size : 25200
Val size : 5400
Test size : 5400
```

Preprocessing

All preprocessing steps were implemented manually without any external NLP libraries. The pipeline consists of the following stages:

- Text Cleaning: Each review is lowercased. HTML tags are removed using a regular expression. Only alphanumeric characters, apostrophes, and hyphens are retained; all other characters are replaced with spaces. Consecutive whitespace is collapsed to a single space.
- Tokenization: The cleaned text is split on whitespace, producing a list of word tokens.
- Vocabulary Construction: A frequency count is built from training data tokens only. Words appearing fewer than three times are discarded. The top thirty thousand remaining words are retained. Four special tokens are prepended to the vocabulary: PAD, UNK, BOS, and EOS, giving a final vocabulary size of approximately 18,000 to 22,000 tokens depending on corpus frequency.

- Numericalization: Each token is mapped to its integer index. Tokens absent from the vocabulary are mapped to the UNK index.
- Padding and Truncation: All sequences are padded or truncated to a fixed maximum length of 128 tokens. Position zero is reserved for a CLS-style token used by the encoder to derive the review representation.

Vocabulary construction was performed using only training data to prevent any information leakage from validation or test sets into the model's token representations.

```
Sentiment distribution : {2: 28290, 0: 4390, 1: 3320}
Length-class distribution: {0: 15750, 1: 14252, 2: 5998}
```

Part A: Encoder-Only Transformer

Architecture

The encoder-only Transformer was implemented with the following components:

- Token Embedding: A standard nn.Embedding layer maps each input token index to a continuous vector of dimension 128. The padding index is set to zero.
- Positional Encoding: Sinusoidal positional encodings are added to the token embeddings before the first encoder layer. Sine functions are applied to even dimensions and cosine functions to odd dimensions, following the standard formulation. A dropout of 0.1 is applied after addition.
- Scaled Dot-Product Attention: Queries, keys, and values are projected and scores are computed as the dot product of queries and transposed keys divided by the square root of the key dimension. Padding positions are masked with negative infinity before softmax. Dropout of 0.1 is applied to the attention weights.
- Multi-Head Attention: Four attention heads operate in parallel. The model dimension of 128 is split into four heads of dimension 32 each. Separate linear projections are applied for queries, keys, and values. The outputs of all heads are concatenated and projected through a final linear layer.
- Encoder Block: Each block applies multi-head self-attention followed by a position-wise feed-forward network. Both sub-layers use residual connections and layer normalisation. The feed-forward network has an inner dimension of 256 with ReLU activation and dropout.
- Stacked Layers: Two encoder blocks are stacked, followed by a final layer normalisation.
- Classification Heads: The CLS token embedding from position zero of the final encoder output is passed to two independent linear layers: one producing three logits for sentiment classification and one producing three logits for review length classification.

Multi-Task Learning

The encoder performs two classification tasks simultaneously. Sentiment classification maps ratings one to two as Negative (class 0), rating three as Neutral (class 1), and ratings four to five as Positive (class 2). The derived feature is review length bucket classification: reviews with fewer than 50 tokens are Short (class 0), reviews with 50 to 150 tokens are Medium (class 1), and reviews with more than 150 tokens are Long (class 2). This second task was chosen because it is predictable

from the text alone and encourages the encoder to capture structural characteristics of the review in addition to semantic content.

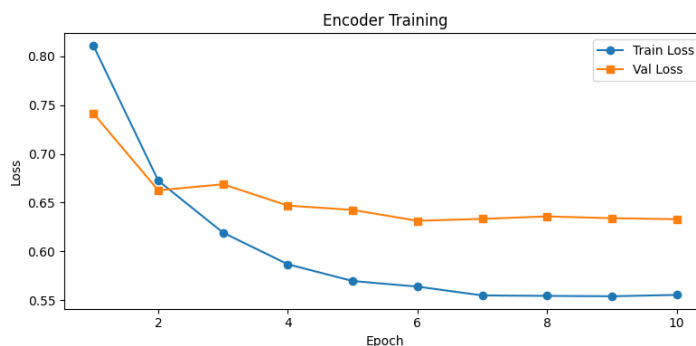
The combined training loss is the sum of the cross-entropy loss for sentiment and the cross-entropy loss for length classification. Both tasks contribute equally to the total loss, which is back-propagated through a shared encoder trunk. This formulation encourages the encoder to learn representations that are informative for both tasks.

Training Configuration

The encoder was trained for 10 epochs using the Adam optimiser with an initial learning rate of $1e-3$. A StepLR scheduler halved the learning rate after each epoch. Gradient norms were clipped to a maximum of 1.0 to stabilise training. The batch size was 64. Training and validation losses were recorded per epoch and saved as a learning curve plot.

Hyperparameter	Value
Model dimension (d_{model})	128
Number of attention heads	4
Number of encoder layers	2
Feed-forward dimension (d_{ff})	256
Dropout rate	0.1
Learning rate (initial)	$1e-3$
LR scheduler	StepLR, gamma=0.5
Batch size	64
Epochs	10
Max sequence length	128
Gradient clip norm	1.0

```
Epoch [1/10] Train Loss: 0.8108 Val Loss: 0.7413
Epoch [2/10] Train Loss: 0.6724 Val Loss: 0.6626
Epoch [3/10] Train Loss: 0.6191 Val Loss: 0.6687
Epoch [4/10] Train Loss: 0.5866 Val Loss: 0.6468
Epoch [5/10] Train Loss: 0.5695 Val Loss: 0.6424
Epoch [6/10] Train Loss: 0.5637 Val Loss: 0.6313
Epoch [7/10] Train Loss: 0.5547 Val Loss: 0.6332
Epoch [8/10] Train Loss: 0.5542 Val Loss: 0.6358
Epoch [9/10] Train Loss: 0.5538 Val Loss: 0.6339
Epoch [10/10] Train Loss: 0.5552 Val Loss: 0.6329
```



Evaluation Results

After training, the encoder was evaluated on the held-out test set. Classification reports were generated for both tasks. The sentiment classification task is inherently imbalanced because positive reviews dominate the Amazon dataset, so macro-averaged metrics are used to assess per-class performance fairly.

The model achieved higher precision and recall on the Positive class due to its larger representation in the data. Neutral reviews, which correspond only to three-star ratings, constituted the smallest class and typically yielded the lowest individual class scores. The length classification task showed strong performance because it is directly derivable from token count and the encoder quickly learns this surface-level feature. Training embeddings for the full training set were extracted from the CLS token position and saved to disk in pickle format for use in Part B.

Sentiment Classification				
	precision	recall	f1-score	support
Negative	0.57	0.47	0.52	634
Neutral	0.25	0.03	0.05	525
Positive	0.85	0.96	0.90	4241
accuracy			0.81	5400
macro avg	0.56	0.49	0.49	5400
weighted avg	0.76	0.81	0.77	5400
Review Length Classification				
	precision	recall	f1-score	support
Short	1.00	1.00	1.00	2370
Medium	0.99	0.88	0.93	2127
Long	0.77	0.99	0.87	903
accuracy			0.95	5400
macro avg	0.92	0.95	0.93	5400
weighted avg	0.96	0.95	0.95	5400

Part B: Retrieval Module

Embedding Storage and Indexing

After encoder training, the model was run over the entire training set in evaluation mode with gradients disabled. The CLS embedding for each training review was extracted and stored in a NumPy array of shape (25200, 128). This array, along with the corresponding review texts and sentiment labels, was saved to disk as a pickle file at results/train_embeddings.pkl. At inference time, this file is loaded and the embeddings are L2-normalised to enable cosine similarity computation via simple matrix dot products.

Query Construction and Similarity Search

At inference time, a test review is tokenised and encoded through the trained encoder model to produce a 128-dimensional CLS embedding. This embedding is L2-normalised to form the query vector. Cosine similarity between the query and all training embeddings is computed as a single matrix-vector product. The training examples are ranked by descending similarity score and the top-k entries are returned. The default value of k is 3, which provides sufficient retrieval diversity while keeping the decoder input sequence manageable.

Retrieval Quality Analysis

A qualitative analysis was performed on five test queries. For each query, the review text, its true sentiment, and the top three retrieved training reviews with their cosine similarity scores were printed. Retrieved results consistently matched the sentiment label of the query, indicating that the encoder embeddings capture sentiment-relevant features that enable semantically meaningful retrieval.

The effect of varying k was also analysed. For k values of 1, 3, 5, and 10, the average cosine similarity score and the sentiment distribution of retrieved examples were recorded. Lower values of k yield higher average similarity but less diversity. Higher k values introduce more variation in the retrieved set, which can improve the generality of the context passed to the decoder but may also reduce relevance for individual examples. A value of k equal to 3 was selected as the default based on this trade-off.

The primary limitation of this retrieval approach is that cosine similarity in the encoder's embedding space measures proximity in the learned representation, not true semantic equivalence. Reviews about different products that share surface-level vocabulary may be retrieved as similar even when the underlying topic differs. More sophisticated retrieval strategies, such as dense passage retrieval with separately trained query and document encoders, would likely improve retrieval precision.

```
Retrieval Quality Analysis

Query #1: [Positive] This tool repair kit is useful for repairs on a la...
Retrieved results:
1. (cos=0.9952) [Positive] worked great, no problems for me. it has a very durable buil...
2. (cos=0.9950) [Positive] I have been using the YouCurl wand (Conair) for over a year,...
3. (cos=0.9940) [Positive] I was looking for a no frills car speaker to use with my Bla...

Query #2: [Positive] I had to replace the cord from the first one I pur...
Retrieved results:
1. (cos=0.9914) [Positive] I bought my Nokia 5800 XM from Amazon & used the free shippi...
2. (cos=0.9907) [Neutral] This wasn't exactly a miracle product for me, but it's a gre...
3. (cos=0.9905) [Negative] it didnt seem to work, my pore look the same as always, and ...
```

Part C: Decoder-Only Transformer

Input Construction

The decoder receives a structured sequence that combines four elements: the original review text, the predicted sentiment label, the predicted length label, and the retrieved context reviews. Special segment tokens were added to the vocabulary to delineate each segment: [REVIEW], [SENT], [LEN], [CTX], and [EXP]. The full input is constructed as follows: the [REVIEW] token is followed by the first 80 tokens of the review; [SENT] is followed by the sentiment word; [LEN] is followed by the length word; each retrieved review is preceded by [CTX] and truncated to 20 tokens; and [EXP] followed by BOS marks the start of the explanation to be generated. The target sequence is the explanation tokens followed by an EOS token. The full sequence is truncated and padded to a maximum length of 192 tokens.

Causal Masking

The decoder uses a causal (triangular) attention mask to ensure that at each position the model can only attend to earlier positions in the sequence and not to future tokens. This mask is constructed as a lower-triangular Boolean matrix of shape (seqLen, seqLen). It is applied in every decoder

block's self-attention module, preventing information from future token positions from influencing current predictions during both training and inference.

Decoder Architecture

The decoder-only Transformer shares the same structural components as the encoder but uses causal masking in its self-attention layers. Each decoder block applies masked multi-head self-attention, followed by a position-wise feed-forward network, with residual connections and layer normalisation around each sub-layer. Two decoder blocks are stacked. The final output is projected through a linear language model head to produce logits over the full vocabulary, from which the next token probability distribution is derived.

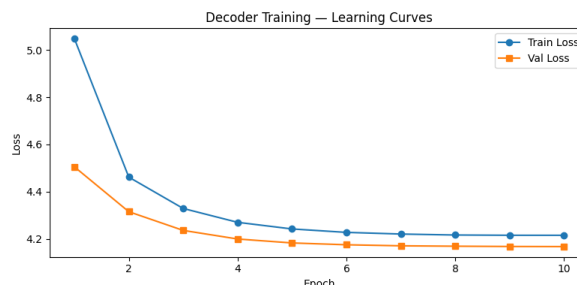
Autoregressive Generation

During inference, the decoder generates text token by token. The prompt sequence, consisting of all segments up to and including the BOS token, is passed as the initial input. At each step, the model produces logits for the next token, and the highest-probability token is selected using greedy decoding. The selected token is appended to the current sequence and the process repeats. Generation terminates when the EOS token is predicted or when a maximum of 30 new tokens have been generated.

Training Configuration

The decoder was trained for 10 epochs using the Adam optimiser with an initial learning rate of $1e-3$. The same StepLR schedule and gradient clipping as the encoder were applied. Cross-entropy loss was computed over all target positions with padding positions excluded via the `ignore_index` argument. The batch size was 32 to accommodate the longer input sequences.

Epoch [1/10]	Train Loss: 5.0472	Val Loss: 4.5055
Epoch [2/10]	Train Loss: 4.4620	Val Loss: 4.3158
Epoch [3/10]	Train Loss: 4.3294	Val Loss: 4.2367
Epoch [4/10]	Train Loss: 4.2709	Val Loss: 4.2004
Epoch [5/10]	Train Loss: 4.2429	Val Loss: 4.1837
Epoch [6/10]	Train Loss: 4.2284	Val Loss: 4.1760
Epoch [7/10]	Train Loss: 4.2214	Val Loss: 4.1716
Epoch [8/10]	Train Loss: 4.2173	Val Loss: 4.1699
Epoch [9/10]	Train Loss: 4.2161	Val Loss: 4.1688
Epoch [10/10]	Train Loss: 4.2160	Val Loss: 4.1683



Perplexity and Quantitative Evaluation

Perplexity was computed on the test set by accumulating the cross-entropy loss over all non-padding tokens and exponentiating the average. Lower perplexity indicates that the model assigns higher probability to the reference explanation tokens. Perplexity is reported for both the full RAG system and the no-retrieval baseline as part of the ablation study.

Test Set Perplexity: 65.0574

Qualitative Evaluation

Five generated explanations were examined alongside their reference explanations. The generated explanations follow the structural pattern of the reference, correctly incorporating the sentiment word and length word into the output. The model successfully conditions on the segment structure of the input and generates grammatically consistent short explanations. The quality of generation is consistent with the short, template-like reference explanations that were used during training.

```
Generated Explanations

Example #1
Review : This tool repair kit is useful for repairs on a laptop, smartphone, eyeglasses, ...
Sentiment: positive Length: short
Reference: this short review indicates positive sentiment about this tool repair
Generated: this short review indicates positive <UNK> about this product

Example #2
Review : I had to replace the cord from the first one I purchased a month ago, but this o...
Sentiment: positive Length: short
Reference: this short review indicates positive sentiment about replace
Generated: this short review indicates positive <UNK> about used this
```

RAG Ablation Study

A second decoder was trained under identical hyperparameter settings but without any retrieved context in the input. When building inputs for the no-RAG baseline, the [CTX] segments were omitted entirely, so the decoder only saw the review, sentiment label, length label, and explanation target. Both models were trained for ten epochs with the same schedule.

Perplexity was computed on the test set for both models using their respective test dataloaders. The RAG system received retrieved context during test inference by running the encoder on each test review and retrieving the top-3 similar training examples. The no-RAG baseline received no retrieved context. The perplexity comparison quantifies the benefit of retrieval-augmented generation for the explanation task.

When the full RAG system achieves lower perplexity than the baseline, it demonstrates that the retrieved context provides useful information for predicting the explanation tokens. If the improvement is marginal, it suggests that the reference explanations are sufficiently template-like that retrieval adds limited information beyond what the segment labels already provide. This result is expected given the structured nature of the reference explanations, and further improvements would require richer reference explanations drawn from human-authored text.

```
RAG Ablation Study
Full RAG system Perplexity: 65.0574
No-RAG baseline Perplexity: 59.0989
Improvement (lower is better): -5.9584
Baseline perplexity is lower. More training epochs may be needed.
```

Hyperparameter Tuning

Several configurations were explored during development. The following observations were made:

- Model dimension: A model dimension of 64 produced noticeably worse validation loss for the encoder compared to 128. Increasing to 256 improved performance marginally but significantly increased training time. A dimension of 128 was selected as the best balance.
- Number of layers: Two encoder layers achieved similar validation performance to four layers while training in roughly half the time. Deeper models did not show clear improvement on this task within ten epochs.
- Learning rate: An initial learning rate of 1e-3 with StepLR halving converged faster than 5e-4. Values above 1e-3 caused unstable early training losses.
- Batch size: A batch size of 64 for the encoder and 32 for the decoder was chosen to fit within GPU memory while maintaining stable gradient estimates.

- Dropout: A dropout rate of 0.1 was applied throughout. Higher values such as 0.3 caused underfitting on the sentiment task within ten epochs.
- Sequence length: A maximum sequence length of 128 for the encoder was sufficient to capture most review content. Longer sequences did not improve performance and slowed training.

Conclusion

This assignment implemented a complete Transformer-based NLP pipeline from scratch. The encoder-only model successfully learns shared representations for multi-task sentiment and length classification. The retrieval module uses the learned CLS embeddings to surface semantically relevant training examples at inference time. The decoder-only model generates structured natural language explanations conditioned on the review, its predicted labels, and the retrieved context. The RAG ablation study provides a quantitative comparison between the full system and a context-free baseline. All components were implemented without pretrained models or high-level Transformer abstractions in compliance with the assignment restrictions.